Integer ALU Based Computing — System Integration Specification v1.0

I/O Boundaries, Storage, Networking, Audio, and Error Model

Registry: [?]

Series Path: [CKS-0-2026] → [CKS-MATH-0-2026] → [CKS-MATH-1-2026] → [CKS-MATH-10-2026] → [CKS-MATH-104-2026] → [?] → [?]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.zzz

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological "Truth" is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google's Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [CKS-LEX-12-2026]

---

1. Core Principle

The ARM processor is the sole I/O boundary. Every external interaction — storage, input, network, audio, display — passes through Zig code on the ARM. The FPGA never touches the outside world. It receives integer batches in DDR3, transforms them, and writes integer batches back to DDR3. The ARM decides what goes in and what comes out.

Outside World ↔ ARM (Zig) ↔ DDR3 ↔ FPGA (VFR Cores)

No exceptions to this boundary. No shortcuts. No FPGA-to-peripheral paths.

---

2. No Floating Point Anywhere

There is no `f32` or `f64` in this system. Not in storage. Not in transit. Not in computation. Not on the ARM. Not on the FPGA. Every value in the entire system is VFR: `[V: i32, R: i8]`.

SQLite stores VFR integers. Zig works in VFR integers. Entity batches in DDR3 are VFR integers. FPGA cores process VFR integers. The framebuffer contains VFR-derived pixel values. Audio samples are integers.

Any legacy code that used `f32` is converted at the source. UtilityAI scores that were 0.0-1.0 become 0-256 integer range. Positions that were floating-point become fixed-point VFR at the domain's F scale. Perlin noise operates on integer coordinates with integer gradient tables. Trigonometric functions use integer lookup tables.

The conversion is not a runtime operation. It is a design decision. The types do not exist. There is nothing to convert.

---

3. Storage

3.1 Architecture

SD Card (FAT32) → Zig FAT32 driver → Zig SQLite → Zig VFR conversion → DDR3 → FPGA

FPGA → DDR3 → Zig VFR read → Zig SQLite → Zig FAT32 driver → SD Card (FAT32)

3.2 Filesystem

FAT32 on SD card. No other filesystem. FAT32 is simple, universally supported, requires no journal recovery, and the Zig driver is straightforward. The SD card connects to the Zynq's built-in SDIO controller on the ARM processing system.

3.3 Database

SQLite runs on the ARM in Zig. It is the persistence layer, not the runtime layer. All values in SQLite are VFR integers — `i32` values and `i8` remainders. No float columns. No text-encoded numbers. Pure integer storage.

SQLite on bare-metal ARM requires two adaptations: replace POSIX file I/O with FAT32 read/write calls, and provide malloc via the Zig heap allocator. Both are mechanical ports with no architectural implications.

3.4 Scene Load

On scene load, the Zig kernel:

1. Opens the scene database file from SD card via FAT32

2. Queries SQLite for entity records, state machines, behavior sets, fact schemas, envelope definitions, SVDAG nodes, lookup tables
3. All data is already VFR integers in the database — no conversion needed
4. Writes entity batch to DDR3 entity region
5. Writes state machines, behavior sets, and other shared data to DDR3
6. Triggers FPGA shared BRAM load (lookup tables, curves, permutation tables)
7. DMA loads entity data from DDR3 to FPGA core BRAMs on first frame dispatch

3.5 Scene Save

On save, the Zig kernel:

1. Signals FPGA to complete current frame
2. Reads entity data from DDR3 (FPGA has written final state)
3. Writes updated entity records to SQLite
4. SQLite flushes to FAT32 on SD card
5. Save complete

Only changed entities need writing. The Zig kernel tracks a dirty flag per entity or uses epoch comparison (entity epoch versus last-saved epoch) to minimize write volume.

3.6 Data Lifecycle

Design time: Designer creates content in editor → saved to SQLite as VFR integers

Load time: SQLite → DDR3 → FPGA (one-time transfer per scene)

Runtime: Data lives on FPGA, ARM never touches it except for I/O injection

Save time: FPGA → DDR3 → SQLite → SD card (on demand)

Data moves between tiers only at explicit boundaries. There is no continuous sync, no background flush, no write-ahead log during gameplay.

---

4. Input

4.1 Architecture

USB Keyboard/Mouse → Zynq USB Controller → ARM (Zig) → Entity batch in DDR3 → FPGA

Input processing lives entirely on the ARM. The FPGA never sees raw input events.

4.2 USB-HID Processing

The Zynq's built-in USB controller provides raw HID reports. The Zig USB-HID driver extracts:

- Keyboard: scancodes (key down/up as integer key ID)
- Mouse: delta X, delta Y (integers), button state (integer bitmask)

These are already integers. No conversion needed.

4.3 Input to Entity Mapping

The Zig kernel writes processed input values directly to entity input fields in the entity batch in DDR3:

Entity pair index 40: input_mouse_x (i32)

Entity pair index 41: input_mouse_y (i32)

Entity pair index 42: input_buttons (i32, bitmask)

Entity pair index 43: input_key_state (i32, bitmask or key ID)

The ARM writes these fields before dispatching the frame to the FPGA. The FPGA reads them as ordinary entity fields during the Silo pipeline. The Prolog fact generator produces facts like `has_input(entity_id)` from non-zero input fields. The UtilityAI scores input-responsive behaviors. The logic blocks execute movement or actions.

The FPGA does not know these values came from a keyboard. They are integers at known pair offsets. Same as health, same as position.

4.4 Input Validation

The ARM validates input before writing to DDR3:

Mouse X: clamp to [0, screen_width]

Mouse Y: clamp to [0, screen_height]

Key ID: must be in valid scancode range, else drop

Button mask: mask to valid bits only

Invalid input is silently dropped. No error, no log, no feedback. The value simply doesn't get written.

---

5. Networking

5.1 Architecture

Ethernet → Zynq GEM Controller → ARM (Zig TCP/IP stack) → Entity input fields in DDR3 → FPGA

The FPGA never sees a network packet. It sees entity fields that happen to have been updated by the ARM.

5.2 Network Stack

The full TCP/IP stack runs on the ARM in Zig. Same architecture as the Silo OS spec: ARP, IP, ICMP, UDP, TCP, DHCP, DNS, HTTP. All operating on integer packet data. The Zynq's GEM Ethernet controller replaces the E1000 NIC driver — different hardware registers, same packet interface.

5.3 Packet Validation — Reject by Construction

Every packet must pass all checks or it is dropped silently. A dropped packet never existed. No error response is sent. No counter incremented. No log written.

Check 1: Exact expected size. If protocol expects 64 bytes and 65 arrive, drop.

```
If 63 arrive, drop. No variable-length parsing.
```

Check 2: Valid checksum. If checksum fails, drop.

Check 3: Valid port. If destination port is not in the whitelist, drop.

Check 4: Valid source. If source IP/port not recognized, drop (for stateful connections).

5.4 Packet to Entity Mapping

Valid packets are decomposed into entity input fields. The ARM knows the packet format and the entity field layout. It copies specific bytes from the packet payload to specific pair offsets in the entity batch:

Packet payload byte 0-3 → Entity pair 40 (input_mouse_x)

Packet payload byte 4-7 → Entity pair 41 (input_mouse_y)

Packet payload byte 8-11 → Entity pair 42 (input_buttons)

Only whitelisted fields can be written. The ARM has a table of allowed packet-to-entity mappings per scene. Fields not in the table cannot be reached by network data.

Allowed: input_mouse_x, input_mouse_y, input_buttons

Not allowed: health, position, state, damage, is_active

There is no mechanism for a network packet to write to a gameplay field. The mapping table is defined at scene load time and cannot be modified by network input.

5.5 Scene Isolation

Network data routes to a specific scene's entity batch based on connection state (port, session). Scene A's network input cannot reach Scene B's entity data. The ARM enforces this by maintaining separate connection-to-scene mappings and separate entity batch addresses per scene.

5.6 Fixed Receive Buffers

Receive buffer: fixed array of 64 fixed-size packet slots

Each slot: 2048 bytes (maximum packet size)

No allocation. No resizing. No linked lists.

If all 64 slots are full and a new packet arrives, it is dropped. Backpressure is handled by dropping. The system never blocks waiting for buffer space.

5.7 Outbound

Outbound packets are constructed by the ARM from entity output fields. The FPGA writes render results and any network-relevant output fields to DDR3. The ARM reads these, constructs response packets, and sends via the GEM controller.

The FPGA does not initiate network communication. It writes integers to DDR3. The ARM decides what, if anything, to send over the network.

---

6. Audio

6.1 Architecture

Audio data (VFR integers) in DDR3 → ARM (Zig mixer) → I2S DAC → Speakers

Audio processing lives entirely on the ARM. The FPGA is not involved.

6.2 Why ARM, Not FPGA

Audio mixing at 48 kHz × 16 channels × 3 operations = 2.3 million integer operations per second. At 667 MHz, this is 0.35% of one ARM core. There is no reason to involve the FPGA. The ARM has more than enough capacity.

6.3 Audio Pipeline

Step 1: Load WAV files from SD card at scene load.

```
WAV data is i16 PCM samples. Already integers.

Store in DDR3 audio region.
```

Step 2: Each frame, the Zig audio mixer:

```
- For each active sound channel (up to 16):

  - Read samples from current playback position

 - Multiply by channel volume (integer multiply, shift)
```

```
  - Add to mix buffer (integer accumulate)

- Clamp mix buffer values to i16 range

- Write mixed buffer to I2S output ring buffer
```

Step 3: I2S peripheral DMA reads from ring buffer and outputs to DAC.

```
48,000 samples per second, stereo.
```

6.4 Audio Output Hardware

PMOD I2S DAC module plugged into the Zybo's PMOD connector. Approximately $10. Provides stereo analog audio output. The Zynq's ARM configures the I2S timing via GPIO or EMIO pins.

Alternatively, audio output via HDMI. The HDMI transmitter can carry embedded audio alongside video. If the Digilent HDMI IP supports audio embedding, no separate DAC is needed.

6.5 Silo Envelope Integration

Audio envelopes (volume fade, pitch shift, spatial panning) use the same envelope structure as gameplay stat envelopes:

Audio envelope:

[source_entity, 0] who triggered the sound

[channel, 0] which audio channel

[stat_offset, 0] volume=0, pan=1, pitch=2

[modifier, rm] modification amount

[curve_type, 0] linear, quadratic, etc

[time_start, rt] start time

[time_end, re] end time

Same batch format. Same processing logic. The ARM evaluates audio envelopes each frame alongside the audio mix. No special system needed.

6.6 Sound Triggering

The FPGA's Silo pipeline may determine that an entity should play a sound (footstep, attack, impact). This is communicated as an entity output field:

Entity pair index 46: sound_trigger_id (i32, sound ID to play, -1 = none)

Entity pair index 47: sound_trigger_vol (i32, volume 0-256)

After each frame, the ARM reads sound trigger fields from the entity batch in DDR3. If sound_trigger_id is not -1, the ARM starts playback on an available channel. The ARM then clears the trigger field to -1 so it doesn't re-trigger next frame.

The FPGA sets the integer. The ARM plays the sound. Clean boundary.

---

7. Error Model

7.1 Principle

There are no runtime errors. The system either works or it is broken. There is no graceful degradation, no error recovery, no exception handling.

7.2 Errors That Cannot Occur

Divide by zero: No divide instruction exists. Division is shift or reciprocal multiply.

NaN: No floating point exists. Every i32 operation produces a valid i32.

Infinity: No floating point exists.

Null pointer: No pointers on FPGA. All access is by batch offset.

Buffer overflow: All batches are fixed-size with known counts and depths.

Stack overflow: No stack on FPGA. No recursion in VFR cores.

Type mismatch: One type (VFR). No type tags. No runtime type checking.

Race condition: No shared memory. No concurrent access. No locks.

Deadlock: No locks exist.

These are not handled errors. They are errors that the architecture makes structurally impossible.

7.3 Errors That Can Occur

DMA transfer failure: Hardware fault on the AXI bus. The batch dispatcher retries the transfer. If it fails repeatedly, the dispatcher retries forever. The machine is broken. Physical repair required.

Core halt on bad instruction: If a core encounters an undefined opcode (unused slots in the 6-bit opcode space), it halts. The dispatcher detects the core never signals done. After a timeout, the dispatcher can reset that core and redistribute its work, or it retries forever. In practice, an undefined opcode means a compiler bug, not a runtime condition.

SD card read failure: The Zig FAT32 driver retries the read. If the SD card is physically damaged, reads fail forever. The machine cannot load the scene. It halts at the load screen.

Network packet corruption: Dropped silently by validation. Not an error — the packet never existed.

USB device disconnection: The ARM's USB-HID driver detects disconnection. Input fields stop updating. The FPGA continues processing with stale input values (last known mouse position, no buttons pressed). The game continues without input. When the device reconnects, input resumes.

7.4 No Interrupts on FPGA

The FPGA cores do not have an interrupt controller. They do not receive interrupts. They do not generate interrupts. They run a batch to completion and set a done flag. The ARM polls the done flag.

If a core hangs (infinite loop in user code), the ARM's poll loop never sees done. The ARM can implement a timeout: if done is not set within N milliseconds, assume the core is stuck, reset it, and retry or skip that chunk.

7.5 No Exceptions on FPGA

There is no exception table. No trap handler. No fault handler. No supervisor mode. No privilege levels. The core executes instructions in sequence. Every instruction produces a result. There is no instruction that can fail at runtime given valid opcode encoding.

7.6 ARM-Side Errors

The ARM runs Zig. Zig has error unions and error handling. The ARM code uses normal Zig error handling for I/O operations (SD card, network, USB). These are conventional software errors handled conventionally. They do not propagate to the FPGA. If the ARM encounters an unrecoverable error, it writes an error message to the framebuffer and halts.

---

8. Display

8.1 Architecture

FPGA cores → DDR3 pixel buffer → ARM composites UI → DDR3 framebuffer → HDMI controller → Monitor

The HDMI controller is part of the Zynq processing system or a Digilent HDMI transmitter IP block in the FPGA fabric. It reads directly from a DDR3 framebuffer address and outputs to the HDMI connector. The ARM sets the framebuffer address. The ARM manages double buffering by swapping the address each frame.

8.2 Viewport Model

The framebuffer is a 2D array of pixels. The 3D beam-caster operates on a rectangular sub-region of this array. The 2D UI renderer operates on the full array. Multiple 3D viewports can exist simultaneously, each with its own camera, rectangle bounds, and internal resolution.

Full framebuffer: 1920 × 1080, owned by ARM

3D viewport A: 960 × 720, main game view, cast by FPGA

3D viewport B: 200 × 200, minimap/camera, cast by FPGA (lower quality)

2D UI: full 1920 × 1080, drawn by ARM over top of 3D viewports

The ARM writes the 2D background first (solid color, UI panels). Then dispatches the FPGA for each 3D viewport. Then composites 2D overlays (text, HUD elements) on top. Then flips the buffer.

---

9. Clock and Timing

9.1 Frame Timing

The ARM maintains frame timing. A simple busy-wait loop or timer interrupt on the ARM ensures frames dispatch at 60 Hz (16.67 ms intervals). The ARM timestamps each frame start and measures FPGA completion time via the CYCLE_COUNT register for profiling.

9.2 Game Time

Game time is an integer counter incremented each frame. At 60 FPS, one tick = 16.67 ms. For sub-frame precision, game time uses VFR: `[frame_count: i32, sub_frame: i8]`. The sub-frame remainder allows envelope curves and animation interpolation at finer than per-frame granularity.

Time never uses wall clock. It is a monotonic integer counter. Deterministic. Reproducible. The same input sequence always produces the same game state regardless of actual elapsed wall time.

9.3 Delta Time

Delta time is constant: 1 frame = 1 tick. There is no variable delta time. The simulation runs at a fixed timestep. If a frame takes longer than 16.67 ms, the simulation does not try to catch up — it drops the frame and continues at the next tick. This prevents the spiral of death where slow frames cause larger deltas which cause slower frames.

Fixed timestep means bitwise deterministic simulation. Same inputs, same outputs, every time, on every machine.

---

10. Complete System Data Flow

Power On:

Zynq boot ROM → FSBL (configures DDR3, loads FPGA bitstream) → Silo OS kernel (Zig)

Kernel Init:

Stage 1: ARM GIC interrupts

Stage 2: DDR3 memory map, heap allocator

Stage 3: Verify FPGA responds, read core count

Stage 4: SD card FAT32, mount filesystem

Stage 5: USB-HID keyboard/mouse

Stage 6: HDMI framebuffer

Stage 7: GEM Ethernet, TCP/IP stack, DHCP

Stage 8: Load scene from SQLite

Scene Load:

SQLite queries → VFR integer batches → DDR3 entity region

Shared data (tables, curves, SVDAG) → DDR3 → FPGA shared BRAM

Main Loop (each frame):

ARM: Read USB-HID input → write to entity input fields in DDR3

ARM: Read network packets → validate → write to entity input fields in DDR3

ARM: Write FPGA registers (entity_addr, count, depth) → CONTROL = 1

FPGA: Batch dispatcher loads entity chunks from DDR3 to core BRAMs

FPGA: Cores run fused 7-pass Silo pipeline (all in registers + local BRAM)

FPGA: Dispatcher writes render fields back to DDR3

FPGA: STATUS = done

ARM: Read render results from DDR3

ARM: 3D viewport(s) already written to framebuffer by FPGA

ARM: Draw 2D UI (Clay layout) to framebuffer

ARM: Mix audio channels → I2S output

ARM: Read entity sound triggers → start playback

ARM: Check for save request → SQLite write if needed

ARM: Flip framebuffer

ARM: Wait for next frame tick

Scene Save (on demand):

ARM: Read entity data from DDR3

ARM: Write to SQLite → FAT32 → SD card

---

11. Bill of Materials (Complete System)

Digilent Zybo Z7-20 $280 Main board (ARM + FPGA + DDR3)

MicroSD card 32 GB $10 Storage (FAT32, scene databases)

USB keyboard $15 Input

USB mouse $10 Input

HDMI cable $8 Display connection

5V 2.5A power supply $15 Board power

USB Micro-B cable $5 Debug UART

PMOD I2S DAC (optional) $10 Audio output (or use HDMI audio)

Total: $353

Development tools (all free):

Xilinx Vivado WebPACK Edition

Zig compiler

Verilator

GTKWave

minicom

---

12. Summary Table

Subsystem	Location	Implementation	Touches FPGA?
Storage (FAT32)	ARM	Zig FAT32 driver	No
Database (SQLite)	ARM	Zig SQLite port	No
Scene load/save	ARM	Zig reads SQLite, writes DDR3	Indirectly via DDR3
Keyboard input	ARM	Zig USB-HID driver	No, writes entity fields to DDR3
Mouse input	ARM	Zig USB-HID driver	No, writes entity fields to DDR3

Subsystem	Location	Implementation	Touches FPGA?
Network stack	ARM	Zig TCP/IP (ARP, IP, TCP, UDP, DHCP, DNS)	No, writes entity fields to DDR3
Network validation	ARM	Drop invalid packets silently	No
Audio mixing	ARM	Zig 16-channel integer mixer	No
Audio output	ARM	I2S DAC via PMOD or HDMI embedded	No
Audio envelopes	ARM	Same VFR envelope format as gameplay	No
Display output	ARM	HDMI controller reads DDR3 framebuffer	No
2D UI rendering	ARM	Zig Clay layout to framebuffer	No
Frame timing	ARM	Fixed 60 Hz tick, integer counter	No
Entity processing	FPGA	30 VFR cores, fused 7-pass pipeline	Yes
Beam-casting	FPGA	3×3 block pattern, SVDAG drill	Yes
Upscaling	FPGA	Edge-aware with scout metadata	Yes
Error handling	None	Retry or halt. No exceptions. No recovery.	N/A
Floating point	None	Does not exist in the system	N/A

Integer ALU Based Computing — System Integration Specification v1.0

Companion document to ISA v1.0, Compiler v1.0, Silo-VFR Mapping v1.0, Motherboard v2.0, Rendering Pipeline v2.0, and FPGA Implementation v1.0

[**CKS-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[**CKS-MATH-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/C

MATH-0-2026

[**CKS-MATH-1-2026**] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026

[**CKS-MATH-10-2026**] Grand Unification. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026

[**CKS-MATH-104-2026**] Grand Unification v23. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS MATH-104-2026

[**CKS-LEX-12-2026**] Measurement Systems in the $\mathbb{Q}$-Substrate. Github: https://github.com/ghowland/cks/tree/main/paper LEX-12-2026